arXiv:2402.07087v3 [cs.LG] 10 Jun 2024

Self-Correcting Self-Consuming Loops for Generative Model Training

Nate Gillman [1] **Michael Freeman** [1] **Daksh Aggarwal** [1] **Chia-Hong Hsu** [1]
Calvin Luo [1] **Yonglong Tian** [2] **Chen Sun** [1 2]

Abstract

As synthetic data becomes higher quality and proliferates on the internet, machine learning models are increasingly trained on a mix of human- and machine-generated data. Despite the successful stories of using synthetic data for representation learning, using synthetic data for generative model training creates "self-consuming loops" which may lead to training instability or even collapse, unless certain conditions are met. Our paper aims to stabilize self-consuming generative model training. Our theoretical results demonstrate that by introducing an idealized correction function, which maps a data point to be more likely under the true data distribution, self-consuming loops can be made *exponentially* more stable. We then propose self-correction functions, which rely on expert knowledge (e.g. the laws of physics programmed in a simulator), and aim to approximate the idealized corrector automatically and at scale. We empirically validate the effectiveness of self-correcting self-consuming loops on the challenging human motion synthesis task, and observe that it successfully avoids model collapse, even when the ratio of synthetic data to real data is as high as 100%.

1. Introduction

Generative models have been used to synthesize training data for various learning tasks, to varying degrees of success. For example, for the tasks of image classification and contrastive representation learning, recent work (Azizi et al., 2023; Tian et al., 2023) finds that using data synthesized from generative models rivals using real data. Unfortunately, there is a gloomier outlook when attempting to generalize this framework to generative model training.

[1]Brown University [2]Google DeepMind. Correspondence to: Nate Gillman <nate_gillman@brown.edu>, Chen Sun <chensun@brown.edu>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024. Copyright 2024 by the authors.

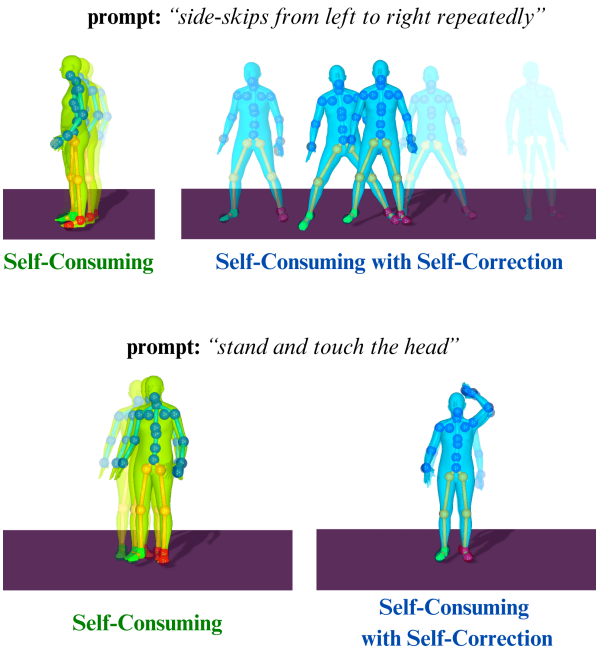

Figure 1. What happens after iteratively training a text-conditioned generative model for human motion synthesis for 50 generations? We simulate a self-consuming loop by creating synthetic data with the latest generative model, and mixing them with the original data to continue training the next generative model. We observe that by self-correcting the synthetic data with a physics simulator, the model can successfully avoid **collapse** and generate **high-quality human motion**. Faded poses represent poses from further back in time. Our paper provides theoretical and empirical justification for the self-correcting self-consuming loop.

On one hand, there is evidence to suggest that training a generative model with its own outputs in a self-consuming manner will lead to collapse (Alemohammad et al., 2024). For example, after 50 iterations of self-consuming training, a human motion diffusion model (Tevet et al., 2023) collapses and fails to follow the text prompts or the laws of physics (see the two examples on the left of Figure 1).

On the other hand, evidence suggests that such a framework could avoid collapse, but only when a "moderate" amount of synthetic data is used (Bertrand et al., 2024). Worse still, this self-consuming scenario might happen without us

knowing, and without us being able to quantify how much synthetic data is being used during training, due to the wide spread of AI generated content on the internet.

Intuitively, model collapse might be delayed or avoided by incorporating higher quality human generated data (Alemohammad et al., 2024), or by manually fixing the "mistakes" in machine created data. Considering the size of datasets used in practice (Schuhmann et al., 2022), neither of these options is a scalable solution.

In this paper, we aim to provide a theoretical analysis of how certain operations would avoid collapse in self-consuming loops, without any assumptions on the "moderateness" of synthetic data corruption. We introduce the mathematical abstraction of a *self-correction operation*. This operation maps synthesized data that are sampled from the generative model to data that are better representatives from the target probability distribution that the model is attempting to approximate. Instead of training on a combination of real data and synthesized data, we propose training on a combination of real data and synthesized *and then self-corrected* data. Note that injecting fresh human generated data can be viewed as a special case of this operation.

Our **main theoretical findings** (Theorem 4.3):

(1) The self-consuming model with self-correction is *exponentially* more stable than the self-consuming model without any self-correction.

(2) The self-correction procedure guarantees less unwanted variance during self-consuming model training.

In our theoretical study, we assume that correction is *ideal* in order to obtain rigorous performance guarantees. In our empirical study, we evaluate whether the same conclusions hold for *noisy* self-correction functions. We propose to automate this "self-correction" process by relying on programmed expert knowledge rather than a human-in-the-loop, such that the function can be applied at scale. We focus on the human motion synthesis task (Guo et al., 2022), and implement the self-correction function with a physics simulator-based imitation model (Luo et al., 2021). Our **empirical results** confirm that our theoretical findings hold in practice:

(1) As illustrated in Figure 1, the self-correcting self-consuming model generates higher-quality human motion than the one without any self-correction.

(2) The self-correction function allows self-consuming loops to avoid collapse even at a high synthetic data to real data ratio (e.g. 100%).

Our theory and experiments suggest that self-correction should stabilize self-consuming model training for any generative modeling task for which there exists a high quality "self-correction" function. We have released all the code associated with this paper.[1]

[1]Project page: https://nategillman.com/sc-sc.html

2. Related Work

2.1. Learning Representations with Synthetic Data

Real curated datasets are costly to obtain, so there has been much interest in generating synthetic data as training data for various vision tasks. Azizi et al. (2023) demonstrates that text-to-image diffusion models such as Imagen (Saharia et al., 2022) can generate synthetic examples that augment the ImageNet dataset for better image classification. He et al. (2023) studies how synthetic data from text-to-image models, when used exclusively, can be used as training data for image recognition tasks. Similarly, Tian et al. (2023) finds that using synthetic outputs from a text-to-image model results in contrastive models whose downstream performance rivals that of CLIP (Radford et al., 2021) on visual recognition tasks, including dense prediction. And the work in Jahanian et al. (2022) explored methods for multi-view representation learning by using the latent space of the generative models to generate multiple "views" of the synthetic data. The above works collectively provide evidence that *some* representation learning tasks, when trained on synthetic data from some *given* generative models, yield excellent results.

2.2. Training Generative Models on Synthetic Data

Another line of reseach investigates the use of synthetic data for training *generative* models. Shumailov et al. (2023) and Martínez et al. (2024) show that the use of model generated content in generative model training results in model degradation, likely because self-consuming loops remove low-density areas from the estimated probability manifold. Alemohammad et al. (2024) formalize three different kinds of self-consuming generative models: the fully synthetic loop, the synthetic augmentation loop, and the fresh data loop. In all of these loops, they iteratively re-train the model from scratch for every new generation. They empirically find that only the fresh data loop avoids model degradation.

Another recent work (Bertrand et al., 2024) considers the problem of iteratively fine-tuning in the context of synthetic augmentation loops. They find that self-consuming augmentation loops do not necessarily collapse, so long as the synthetic augmentation percentage is sufficiently low. The authors use techniques from the field of performative stability (Perdomo et al., 2020) to prove the existence of a convergence phenomenon in the space of model parameters. Our paper differs from prior work as we conduct analysis on self-consuming generative model training when the synthetic data can be optionally *corrected*. The correction can be performed with a human-in-the-loop, or by incorporating learned or programmed expert knowledge, as explored for natural language (Saunders et al., 2022; Welleck et al., 2023; Wu et al., 2023) and human motion (Yuan et al., 2023; Xu et al., 2023). We validate our theory with a practical self-correcting operations designed for image generation

and human motion synthesis tasks.

3. Overall Training Procedure

We describe our proposed procedure in concise language in Algorithm 1, and we explain it in more detail here. We train the zero'th generation from scratch on the ground truth dataset $\mathcal{D}_{\text{real}} := \{x_i\}_{i=1}^n$, and we stop training when the model is close to convergence. For all the following generations, we fine-tune the previous generation's latest checkpoint on a combination of the ground truth dataset $\mathcal{D}_{\text{real}}$, as well as $\lfloor \lambda \cdot n \rfloor$ synthetic data points which are generated from the previous generation's latest checkpoint, and then passed through the *correction function* π_γ.

The correction function π_γ is parameterized by the *correction strength* $\gamma \in \mathbb{R}_{\geq 0}$, which controls how much influence the correction function has on the input data points towards increasing a given point's likelihood with respect to the target distribution. The other main hyperparameter $\lambda \in \mathbb{R}_{\geq 0}$ is the *synthetic augmentation percent*, and it controls the ratio of synthetic data to real data in each iteration of fine-tuning. When $\gamma = 0$, we recover iterative re-training with synthetic augmentation considered in (Bertrand et al., 2024). And if we choose the synthetic augmentation percent to be $\lambda = 0$, then each generation simply corresponds to fine-tuning the model on the same dataset that it was trained on initially.

We now use *iterative fine-tuning* interchangeably with the more general term self-consuming loop. We also consider the idealized correction function for our theoretical analysis, and a broader family of practical *correction functions* for different data types.

4. Theoretical Analysis

4.1. Preliminaries

We mostly follow the notation from (Bertrand et al., 2024), except for introducing the correction function π_γ. Let us denote by p_{data} the ground truth probability distribution that we want to train a generative model to estimate. Suppose we have some dataset $\mathcal{D}_{\text{real}} = \{x_i\}_{i=1}^n$ sampled from p_{data}. We write $\hat{p}_{\text{data}} = (1/n)\sum_{i=1}^n \delta_{x_i}$. More generally, we use a hat to denote the empirical distribution over finitely many samples from the corresponding distribution.

Suppose that we have a class of generative models parameterized by $\Theta \subset \mathbb{R}^d$. We denote by p_θ a probability distribution in this class with model parameters $\theta \in \Theta$. We define the optimal model parameters within this class to be

$$\theta^\star = \arg\max_{\theta' \in \Theta} \mathbb{E}_{x \sim p_{\text{data}}}[\log p_{\theta'}(x)], \tag{1}$$

where we break ties by minimizing $\|\theta^\star\|$. Typically, such optimal parameters yield a model $p_{\theta^\star}$ which closely approximates the oracle ground truth distribution p_{data}, but doesn't equal it exactly; accordingly, we define the Wasserstein-2 distance between the distributions to be

$$\varepsilon := d_W(p_{\theta^\star}, p_{\text{data}}). \tag{2}$$

The model weights for the first generation are naturally defined according to the optimization

$$\theta_0^n := \arg\max_{\theta' \in \Theta} [\mathbb{E}_{x \sim \hat{p}_{\text{data}}}[\log p_{\theta'}(x)]]. \tag{3}$$

This corresponds to training on the finite subset $\mathcal{D}_{\text{real}}$. Next, let us suppose that the model weights from generation t are denoted θ_t^n. We will formalize a procedure for updating these weights for the next generation to obtain θ_{t+1}^n. For this, we need to define our correction function, and then we will use it to define the weight update.

Definition 4.1. For any probability distribution, and for any $\gamma \in \mathbb{R}_{\geq 0}$, we define the *correction of strength* γ of distribution p_θ to be the distribution

$$\pi_\gamma p_\theta(x) := \frac{p_\theta(x) + \gamma p_{\theta^\star}(x)}{1 + \gamma}, \tag{4}$$

where $p_{\theta^\star}$ is defined in (1). For any augmentation percentage $\lambda \geq 0$, we define the *weight update* mapping to be

$$\begin{aligned} \pi_\gamma \mathcal{G}_\lambda^n(\theta) &:= \operatorname*{local\,argmax}_{\theta' \in \Theta} \hat{\mathcal{H}}(\theta, \theta') \\ &:= \operatorname*{local\,argmax}_{\theta' \in \Theta} \Big[\mathbb{E}_{x \sim \hat{p}_{\text{data}}}[\log p_{\theta'}(x)] \\ &\qquad + \lambda \mathbb{E}_{x \sim \widehat{\pi_\gamma p_\theta}}[\log p_{\theta'}(x)] \Big], \end{aligned} \tag{5}$$

where $\hat{p}_{\text{data}}$ and $\widehat{\pi_\gamma p_\theta}$ are empirical distributions of size n and $\lfloor \lambda \cdot n \rfloor$ respectively.

To continue our discussion from before, our iterative weight update is defined as $\theta_{t+1}^n := \pi_\gamma \mathcal{G}_\lambda^n(\theta_t^n)$.

Note that we use an global maximization in (3) when defining the initial parameters θ_0^n, but we use a local maximization when computing our parameter update in (5). This difference is analogous to the differences between how model weights update during initial training, where parameter updates are more global, and during fine-tuning, where parameter updates are more local.

4.1.1. Understanding the correction $\pi_\gamma p_\theta(x)$

For $\gamma = 0$, the correction mapping in (4) simplifies to $\pi_0 p_\theta = p_\theta$, which is just the original distribution; this corresponds to no correction at all. For $\gamma = 1$, it is $\pi_1 p_\theta = (p_\theta + p_{\theta^\star})/2$. And for $\gamma = \infty$, it is $\pi_\infty p_\theta = p_{\theta^\star}$, which corresponds to the optimal distribution. So as γ increases from 0 to ∞, the distribution $\pi_\gamma p_\theta$ has a likelihood profile that matches p_θ less, and $p_{\theta^\star}$ more. As $p_{\theta^\star}$ is the optimal